

SELECTION SORT

Date : 10/07/2025

Topic : Selection Sort

1. INTRODUCTION

Selection Sort is a simple comparison-based sorting algorithm. It works by repeatedly finding the minimum element from the unsorted part of the list and putting it at the beginning of that part.

This process is repeated until the entire list is sorted.

Key Characteristics

- In-place
- Not Stable
- Comparison Sort
- Simple to implement

2. IDEA / INTUITION

- Find the minimum element in the unsorted part.
- Swap it with the first unsorted element.
- Move the boundary of sorted part by one element to the right.
- Repeat until the entire array is sorted.

3. ALGORITHM

1. Start
2. For each position i from 0 to $n-2$
 - Assume the current position i as the minimum.
 - Find the minimum element in the remaining unsorted part ($i+1$ to $n-1$).
 - If a smaller element is found, update the min index.
 - Swap the minimum element with the element at position i .
3. Stop

4. PSEUDOCODE

```
SelectionSort(arr)
n ← length(arr)
for i ← 0 to n-2
    minIndex ← i // Assume current as min
    for j ← i+1 to n-1
        if arr[j] < arr[minIndex]
            minIndex ← j // Update min index
    swap(arr[i], arr[minIndex])
    // Place min at correct position
```

5. EXAMPLE DRY RUN

Let the array be : 29, 10, 14, 37, 13

Pass 1 ($i = 0$) :

29 10 14 37 13 Min = 10
↓ Swap 29 and 10
10 29 14 37 13 10 is in correct position

Pass 2 ($i = 1$) :

10 29 14 37 13 Min = 13
↓ Swap 29 and 13
10 13 14 37 29 13 is in correct position

Pass 3 ($i = 2$) :

10 13 14 37 29 Min = 14
↓ No swap (already correct)
10 13 14 37 29 14 is in correct position

Pass 4 ($i = 3$) :

10 13 14 37 29 Min = 29
↓ Swap 37 and 29
10 13 14 29 37 29 is in correct position

Sorted array : 10, 13, 14, 29, 37

6. IMPORTANT NOTE

- * At every pass, one element is placed at its correct position, (the minimum of unsorted part). So, after i -th pass, the first $i+1$ elements are sorted.

7. TIME & SPACE COMPLEXITY

Case	Time Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

(Time complexity is $O(n^2)$ in all cases)

8. ADVANTAGES & DISADVANTAGES

Advantages

- ✓ Simple and easy to understand.
- ✓ In-place sorting (uses $O(1)$ extra space).
- ✓ Performs same number of comparisons in all cases.

Disadvantages

- ✗ Not stable (relative order of equal elements may change).
- ✗ Not efficient for large datasets ($O(n^2)$ time).
- ✗ Does more swaps than necessary.

Best suited for small arrays or when memory write is costly.



* SELECTION SORT - PAGE 2 *

Date : 10 / 07 / 2025

Topic : Selection Sort

1. EXAMPLE DRY RUN (CONTINUED)

Let the array be : 29, 10, 14, 37, 13

Pass 4 ($i = 3$) :

10	13	14	37	29
----	----	----	----	----

 Min = 37

↓ Swap 37 and 29

10	13	14	29	37
----	----	----	----	----

 37 is in correct position

Array is now sorted.

Sorted array : [10, 13, 14, 29, 37]



In each pass, the minimum element is selected from the unsorted part and placed at the beginning of that part.

2. ACTUAL CODE (C)

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) { // One by one
        // move boundary of unsorted part
        int minIndex = i; // Assume current
                           // index as minimum
        for (int j = i + 1; j < n; j++) {
            // Find the minimum element
            if (arr[j] < arr[minIndex])
                minIndex = j; // Update min index
        }
        if (minIndex != i) { // Swap if needed
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}

int main() {
    int arr[] = {29, 10, 14, 37, 13};
    int n = sizeof(arr) / sizeof(arr[0]);
    selectionSort(arr, n);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

How it works?

- Find min element in unsorted part
- Swap with first element of unsorted part
- Shrink unsorted part and repeat

3. VISUAL SUMMARY

Pass	Array	Min Found	Swap	Result
0	29 10 14 37 13	10	29 ↔ 10	10 29 14 37 13
1	10 29 14 37 13	13	29 ↔ 13	10 13 14 37 29
2	10 13 14 37 29	14	No swap	10 13 14 37 29
3	10 13 14 37 29	29	37 ↔ 29	10 13 14 29 37

4. COMPLEXITY ANALYSIS

- Best Case (Already Sorted) : $O(n^2)$
- Average Case : $O(n^2)$
- Worst Case (Reverse Sorted) : $O(n^2)$
- Space Complexity : $O(1)$

Why $O(n^2)$?

We have two nested loops:

Outer loop runs $(n-1)$ times

Inner loop runs $(n-i-1)$ times in i^{th} iteration

Total comparisons = $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$

Which is $O(n^2)$.

5. PROPERTIES

- In-place algorithm (uses $O(1)$ extra space).
- Not stable (relative order of equal elements may change).
- Performs same number of comparisons in all cases.
- Minimum number of swaps → at most $(n-1)$ swaps.

6. ADVANTAGES

- ✓ Simple and easy to implement.
- ✓ Works well for small datasets.
- ✓ Requires minimum number of swaps.
- ✓ In-place (no extra memory).

7. DISADVANTAGES

- ✗ Not efficient for large datasets ($O(n^2)$ time).
- ✗ Not stable.
- ✗ Not adaptive (doesn't take advantage of sorted data).

8. IMPORTANT POINTS (FOR EXAMS/INTERVIEWS)

- * Selection Sort is a comparison-based sorting algorithm.
- * It divides the array into two parts: sorted and unsorted.
- * Repeatedly selects the minimum element from the unsorted part and moves it to the sorted part.
- * Useful when write operations (swaps) are costly.
- * Number of swaps $\leq n - 1$ (good when memory write is expensive).

9. COMMON MISTAKES

- Not updating minIndex correctly.
- Forgetting to swap the minimum with the first element of unsorted part.
- Running inner loop till n instead of $n - i - 1$.



"Simple today, efficient always - that's the idea of Selection Sort!"

